Reinforcement Learning from Human Feedback (RLHF) with CartPole

Laura McDonnell[1] **Luisa Escobar-Robledo**[1] **Ron Sarma**[2] **Tracy Tabib**[1]
[1]University of Pittsburgh [2]Carnegie Mellon University
{lmcdonne, lescobar, rsarma, ttabib}@andrew.cmu.edu

1 Introduction

Reinforcement learning (RL), through trial and error, mimics how humans learn from experience. It is a powerful machine learning technique, used to train software to make optimal decisions. In RL, taking action results in feedback to "reward" or "punish" certain actions with the goal of learning a policy that will maximize reward over time. RL has been utilized to train algorithms to play complex games like chess and Go (as described in [1]) as well as optimize how robots and autonomous vehicles interact with their environment. In spite of its great potential, however, RL faces several significant challenges that are preventing it from being widely adopted in more real-world applications. One of these is that RL algorithms struggle with stability and convergence, often producing unpredictable or inconsistent results [2]. Addressing this issue is important for advancing RL beyond the specific tasks it has been employed in and allowing it to be deployed to solve complex real-world problems.

In particular, certain tasks are easily evaluated by a human but are tough to explicitly define and so remain challenging for machine learning models to optimize. Does a text sound biased? Is the media being watched entertaining? A human can assess this readily and rank “phrase one” as sounding more biased than “phrase two”. Humans can also readily score and rank the entertainment value of “game 1” versus “game 2” or provide a numerical assessment of how much each was enjoyed. This human feedback can be leveraged to more efficiently train machine learning models, rather than having to, for example, curate large datasets of biased and non-biased texts. This is the paradigm that underpins reinforcement learning from human feedback (RLHF), a technique that incorporates human preferences into model optimization.

Human-machine interaction in machine learning holds significant potential, revolving around the collaborative relationship between a program and its human user. Ideally, this interaction should be highly cooperative and user-friendly. However, an inherent challenge is that human users often exhibit suboptimal and unpredictable behaviors, with their preferences difficult to articulate or quantify precisely. This raises a key question: how can a program be designed to optimize an unknown reward function that reflects a human user’s preferences? Our approach leverages reinforcement learning and online training to customize user-specific adaptations of the algorithm based on individual user behavior and preferences. In a controlled experimental setup, these algorithms generate simulations for the game CartPole, rated subjectively by users based on their perception of the simulation. CartPole is a version of the pole-balancing problem and was introduced by Richard Sutton as a problem that is crucial in understanding how brains learn difficult concepts; it accomplishes this task through mimicry of neuron-like adaptive elements. Our project aims at unraveling how subjective rating introduces unpredictable, suboptimal feedback to the CartPole problem. Success is defined by increasing user satisfaction ratings across training iterations, as the algorithm adapts to generate different playing strategies based on user ratings. This study explores the extent to which reinforcement learning can capture and influence user-specific, implicit reward functions in cooperative human-machine interactions.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

2 Summary of Data

This work utilizes Gymnasium which is modern framework designed for reinforcement learning (RL) research [3] . It builds upon the OpenAI Gym but includes greater functionality, compatibility, and ease of use. Gymnasium was chosen as it provides a standardized interface compatible with a wide variety of RL algorithms and environments. The classic control environments of CartPole, specifically, is utilized in this project. Ideally, the use of Gynamasium and CartPole will allow for greater reproducibility and customization.

Usage of CartPole itself is based off the work of Barto, Sutton, and Anderson who, in 1983, introduced adaptive heuristic critic (AHC) and temporal difference (TD) learning algorithms [4]. These algorithms provided early demonstrations of reinforcement learning principles for solving complex control problems. In particular, they made key contributions: neuronlike adaptive elements, a reinforcement learning framework, control problems demonstration, and formalization of gradient descent techniques. The neuronlike adaptive elements functioned as an artificial system that mimics biological neurons, and were thus capable of adaptive learning through interactions with their environment. The reinforcement learning framework was a set of proposed mechanisms where an agent would improve its behavior using feedback signals, unlike supervised learning, where explicit target outputs are provided. Reinforcement learning can thus leverage evaluative feedback to guide learning. CartPole, specifically, was used in control problems demonstration with its pole-balancing being the control task. With CartPole, one can demonstrate how adaptive elements learn efficient control policies. Finally, the authors formalized the use of gradient-descent techniques in reinforcement learning and highlighted the significance of bootstrapping methods like temporal difference learning. Because of all these factors, this paper is considered a cornerstone in reinforcement learning research.

Data collected in this study consisted of subjective user ratings from interactions with individualized versions of the algorithm, each tailored to capture specific user preferences. For each training round, users rated generated simulations within the CartPole arcade environment on a subjective scale, ranging from 0 to 10 (with 0 being least interest while 10 is most interest), reflecting the degree of interest or aesthetic satisfaction they experienced. These ratings served as feedback for the respective algorithm instances, guiding their iterative adjustment to better align with each user's implicit preferences. The dataset includes these ratings across multiple training rounds for each user-algorithm pair, capturing variations in user satisfaction as the algorithm progressively optimizes pattern generation. Additionally, metadata such as training iteration numbers, pattern characteristics, and individual user identifiers were recorded to facilitate analysis of both algorithmic and user-specific trends in preference learning.

In terms of bias, RLHF has unique factors to consider as it relies on users/evaluators to guide and refine the training process. These biases can go on to impact how fair and robust the system is. RLHF is affected by subjectivity as human feedback inherently reflects individual preferences which can also lead to inconsistent scoring. Anchoring bias can also occur if users rely too much on initial examples or prior knowledge, which can skew the provided feedback. Implicit biases and demographic skew in the pool of users can also affect the system. Human feedback can also be unclear or inconsistent, introducing noise into the learning process which can cause feedback noise ambiguity [5]. With this in mind, since the aim of this project is to capture the user-specific and subjective rewards, bias is an inherent part of the model.

3 Methods

An extensive survey of the literature on RLHF yielded several methods like Proximal Policy Optimization (PPO)[6], supervised learning methods such as behavior cloning (BC) [7] and human-labeled reward models, pure reward-based RL methods such as handcrafted reward functions [8] and inverse reinforcement learning (IRL) [9], existing RLHF implementations such as deep RLHF models [5] and fine-tuned GPT models [10]. We chose to develop a hybrid supervised RLHF approach and compared it against REINFORCE as the baseline.

This study performs comparative reinforcement learning with human feedback implementation for reward function optimization. We trained this model traditionally to play CartPole and optimize reaching the "win" state as fast as possible. The agent wins when it can keep the pole upright either by moving left or right. The episode ends when the pole is more than 15 degrees from the

vertical axis, or the cart moves more than 2.4 units from the center to either left or right (see Figure 2). The objective is complete when the agent scores an average of 475 out of 500 points over 100 consecutive episodes. We compare this with the objective of maximizing human satisfaction which is subjectively rated on a normalized scale from 0 to 10. This study employed an online version of reinforcement learning algorithm with human feedback (RLHF) to develop personalized human-machine interaction models. The RLHF algorithm was adapted to iteratively learn from user feedback within a cooperative framework, focusing on optimizing an unknown reward function representing implicit user preferences. To modify this framework, we used explicit human feedback to guide and shape the learning process. A diagram of the approach used in this project is outlined in Figure1. Each

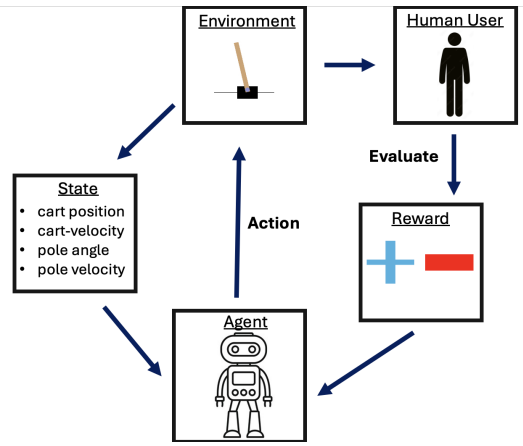

Figure 1: Approach to CartPole RLHF Implementation

user interacted with an individualized instance of the algorithm, which generated simulated gameplay in the CartPole arcade environment, shown in Figure2. Users rated these gameplay patterns on a subjective scale according to their personal interest in the resulting output. The algorithm updated its parameters based on these ratings, with each iteration aiming to enhance its ability to produce simulations that aligned with each user's preferences. Data on user ratings, pattern characteristics, and algorithm iterations were collected and used to evaluate the effectiveness of the model in capturing and adapting to the user's subjective preferences over time. The success of each algorithm instance was measured by an increase in user satisfaction scores across training rounds, indicating improved alignment with the user's implicit reward function. Figure 3 provides pseudocode for implementation of the baseline algorithm versus Figure 4 which includes the pseudocode for the algorithm that incorporates human feedback.

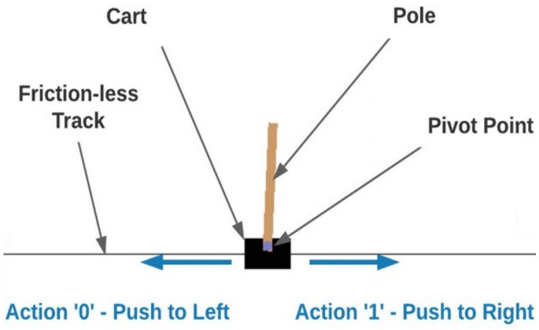

Figure 2: User Interface for Visualizing CartPole Being Played

```
for game_session in range(num_sessions):
    get_trajectory()
    get_policy_loss()
    update_policy()
```

Figure 3: Pseudocode for the baseline algorithm implementing the policy gradient REINFORCE method in which we iterate through each trajectory

```
for game_session in range(num_sessions):
    get_trajectory()
    update_reward_with_human_feedback()
    get_policy_loss()
    update_policy()
```

Figure 4: Pseudocode for REINFORCE with Human Feedback

3.1 Reinforcement Learning

The reinforcement learning algorithm initializes an environment and a policy (π) with random weights. It then runs through a series of episodes (t). In each episode, the environment is reset, and the initial state (s_0) is observed. At each time step within an episode, the agent selects an action (a_t) based on the current policy and executes it. The resulting reward (r_t) and next state are observed, and the policy is updated using the transition information, (includes the current state, action, reward, and next state). This process repeats for a specified number of episodes, allowing the policy to evolve and improve over time.

Algorithm 1 Reinforcement Learning

Initialize environment
Initialize policy π with random weights θ
for episode = 1 to M **do**
    Reset the environment and observe the initial state s_1
    for t = 1 to T **do**
        Select action a_t using the policy $\pi(a_t|s_t)$
        Execute action a_t and observe reward r_t and next state s_{t+1}
        Update the policy π using the observed transition (s_t, a_t, r_t, s_{t+1})
    end for
end for

3.2 Policy Gradient

The policy gradient algorithm differs from the reinforcement learning algorithm in its update policy, but both algorithms aim to improve an agent's behavior through interaction with the environment. The policy gradient algorithm initializes the policy parameters (θ) randomly and iterates through a set of steps where it collects the trajectories by executing the current policy, computes the return for each time step and estimates the policy gradient ($\nabla_\theta J(\theta)$). The policy parameters are then updated using this gradient to maximize the expected return. While the policy gradient algorithm directly optimizes the policy itself, reinforcement learning focuses on learning value functions to guide action selection.

Algorithm 2 Policy Gradient

Initialize policy parameters θ randomly
for iteration = 1 to K **do**
    Collect a set of trajectories $\mathcal{D} = \{(s_t, a_t, r_t)\}$ by executing the current policy π_θ
    Compute the return $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ for each time step
    Compute the policy gradient estimate:
    $\nabla_\theta J(\theta) = \mathbb{E}_{\mathcal{D}} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) G_t \right]$
    Update the policy parameters:
    $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
end for

3.3 REINFORCE

The REINFORCE algorithm, part of the Monte Carlo family of algorithms, is a central policy gradient method used in reinforcement learning; it optimizes an agent's decision-making policy directly and aims to maximize the expected cumulative reward by adjusting policy parameters. [11] This project, specifically, implements the REINFORCE algorithm and uses a scaling factor applied to the return value before computing the policy gradient to introduce the human interest factor[12] [11]. REINFORCE is composed of the steps of policy initialization, episode sampling, trajectory recording, return calculation, policy gradient computation, parameter updating, and iteration. Parameters include the learning rate for the step size of parameter updates during gradient ascent, discount factor to determine the importance of future rewards, policy network architecture, and an optional baseline.

Algorithm 3 REINFORCE Algorithm

1. Initialize the policy π_θ with parameters θ
2. Initialize the value function $V(s)$
3. Initialize an empty buffer for storing rewards and log probabilities
4. **For each episode:**
    (a) Initialize the environment and get the initial state s_0
    (b) Set $t = 0$
    (c) **While not terminal state s_t:**
        i. Select action $a_t \sim \pi_\theta(a_t|s_t)$
        ii. Execute action a_t, observe reward r_t and next state s_{t+1}
        iii. Store (s_t, a_t, r_t) in the buffer
        iv. Set $s_t = s_{t+1}$
        v. Set $t \leftarrow t + 1$
    (d) Compute the returns $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ for each time step t
    (e) **For each time step t in the episode:**
        i. Compute the log probability: $\log \pi_\theta(a_t|s_t)$
        ii. Compute the gradient of the log probability: $\nabla_\theta \log \pi_\theta(a_t|s_t)$
        iii. Update the policy parameters:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

4 Results

The results demonstrate a successful implementation of different reinforcement learning algorithms like REINFORCE and a modified version to incorporate the HF in RLHF at the game of CartPole. CartPole was played on a user-friendly interface in which the states can be visualized such that the human user can subsequently rate the performance of the algorithm in terms of how subjectively satisfying it is. In the baseline implementation, the algorithm optimizes reaching the win state in

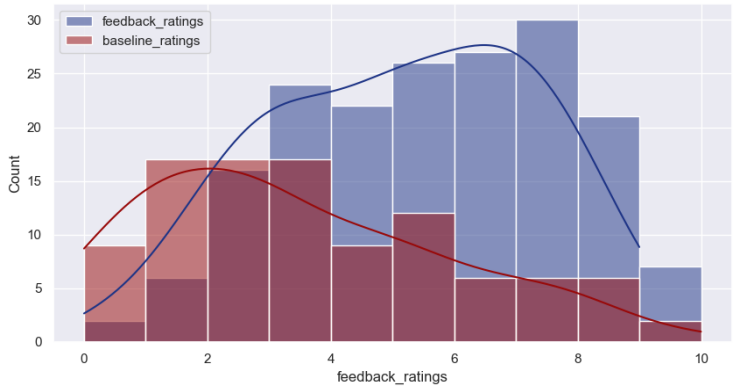

Figure 5: Results of Baseline Algorithm (red) versus REINFORCE Algorithm modified with human feedback (blue). The human user's mean enjoyment rating of the output of the baseline algorithm was 3.396 while the introduction of human feedback raised this to 5.144 for a p-value of 4.916×10^{-9}, computed by t-test

a human-observable manner as its policy goal. The modified algorithm incorporates the human feedback to alter the policy for the optimization of subjective human satisfaction.

These results aligned with expectations that human feedback would provide output that was more pleasing to the user. In Figure 5 we can see that the addition of human feedback increased the mean user score from 3.396 to 5.144, a significant increase in user enjoyment (t-test p-value of 4.916×10^{-9}). The range in the distribution of ratings for outputs of the baseline algorithm in comparison to the implementation with human feedback was not significant, with a baseline variance of 4.735 compared to 5.9216 with human feedback, giving a p-value of 0.09768 by the F-test. Based on this insignificant variance, we can conclude that the range of user ratings was largely consistent, regardless of whether or not the output generated was being produced by an algorithm that incorporated human feedback. But the human feedback did have the effect of shifting the mean user rating significantly higher. As the algorithm adjusted to the human feedback, its output overall became more pleasing and interesting to the human user.

The successful implementation of human feedback in the CartPole reinforcement learning algorithm demonstrates the potential of RLHF in broader AI applications. This approach addresses a critical challenge in AI development: aligning AI performance with human preferences and values. By incorporating subjective human satisfaction into the optimization process, RLHF can create AI systems that not only achieve technical objectives but also produce outputs that are more appealing and useful to human users. The significant increase in mean user scores observed in this study (from 3.396 to 5.144) underscores the effectiveness of this approach. However, the lack of significant change in score variance suggests that RLHF may enhance overall performance without necessarily improving consistency across different scenarios or users. This insight is valuable for future RLHF implementations, highlighting the need to balance improvements in average performance with efforts to reduce variability and ensure consistent user experiences across diverse applications and user groups.

5 Discussion and Analysis

Several assumptions are made regarding the Bayesian reward of the REINFORCE model. The success of this model depends on making the correct prior and likelihood assumptions. If the chosen priors do not accurately represent the human preference or task constraints then the model might not converge to the optimal policy. An incorrect prior could also lead to biased reward estimates that do not adequately recapitulate the human preferences. Too weak or too strong of a prior could overly constrain the model or fail to capture human rating variability. The possibility of overfitting to the provided human feedback also exists, especially if (as is the case in this rather limited project) the data is sparse or inconsistent. Inconsistency is a previously mentioned bias that must be considered when handling human feedback and incorporating it into a model. This overfitting could lead to

suboptimal reward shaping. Another significant limitation is generalizability as this simple reward model is trained on CartPole and may not generalize well to more complex environments or other tasks with different reward structures.

Future directions could include the implementation of more advanced RL algorithms such as the Q-learning algorithm or Deep Q-Network (DQN). These algorithms could enhance performance and adaptability in complex environments.

5.1 Advanced RL Algorithms

5.1.1 Q-Learning Algorithm

Q-learning is a model-free reinforcement learning algorithm that aims to find the optimal action-selection policy for a given environment. The algorithm works by learning the action-value function, $Q(s, a)$, this represents the expected cumulative reward of taking action a in state s and then it follows the optimal policy afterwards.

Algorithm 4 Q-Learning Algorithm

Require: Learning rate α ($0 < \alpha \leq 1$), Discount factor γ ($0 \leq \gamma < 1$), Exploration rate ϵ ($0 \leq \epsilon \leq 1$) and total number of episodes (T)

Ensure: Learned Q-values $Q(s, a)$ and optimal policy $\pi^*(s) = \arg\max_a Q(s, a)$

Initialize $Q(s, a)$ arbitrarily (e.g., $Q(s, a) = 0$ for all s, a)
Set episode $\leftarrow 0$
for episode $= 1$ to T **do**
    Initialize starting state s
    while s is not terminal **do**
        Choose action a based on an epsilon-greedy policy:

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg\max_{a'} Q(s, a') & \text{with probability } 1 - \epsilon \end{cases}$$

        Take action a, observe reward r, and next state s'
        Update Q-value:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big]$$

        Set $s \leftarrow s'$
    end while
end for
return Learned Q-values $Q(s, a)$ and policy $\pi^*(s)$

5.1.2 Deep Q-Network Algorithm

The Deep Q-Network (DQN) algorithm combines Q-learning with deep learning to enable reinforcement learning in high-dimensional state spaces. It begins by initializing a replay memory, an action-value function Q, and a target action-value function $\hat{Q}$ with random weights. The algorithm iterates through multiple episodes, where for each time step, it selects an action based on an ϵ-greedy policy (this can be either random or based on the current action-value function). After executing the action, the algorithm observes the reward and the next state, storing the transition in the replay memory. A random mini-batch of transitions is sampled from memory, and the target value is computed, updating the target network for terminal states or using the maximum action-value for non-terminal states. The model parameters are updated by performing a gradient descent step on the loss between the predicted and target values.

Q-learning could allow for a hybrid approach: combining it with RLHF and allowing agents to learn from both human feedback and their own experiences effectively. DQNs are another option since DQNs use neural networks to approximate Q-values. This allows them to handle high-dimensional state spaces and future work could benefit from incorporating DQNs, potentially through techniques like reward shaping, where human insights refine the reward signals used in DQN training. DQNs also tend to be more sample-efficient than REINFORCE, especially in environments with discrete

Algorithm 5 Deep Q-Network (DQN)

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
for episode = 1 to M **do**
    Initialize state s_1
    for t = 1 to T **do**
        Select action a_t using ϵ-greedy policy:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon, \\ \underset{a}{\text{argmax}}\, Q(s_t, a; \theta) & \text{otherwise.} \end{cases}$$

        Execute action a_t and observe reward r_t and next state s_{t+1}
        Store transition (s_t, a_t, r_t, s_{t+1}) in D
        Sample a random minibatch of transitions (s_j, a_j, r_j, s_{j+1}) from D
        Set target:

$$y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal,} \\ r_j + \gamma \underset{a'}{\max}\, \hat{Q}(s_{j+1}, a'; \theta^-) & \text{otherwise.} \end{cases}$$

        Perform a gradient descent step on loss:

$$L(\theta) = \mathbb{E}\left[(y_j - Q(s_j, a_j; \theta))^2\right]$$

        Every C steps, update target network:

$$\theta^- = \theta$$

    end for
end for

action spaces like CartPole [13]. So effective policies can be learned with fewer interactions and this is particularly valuable when incorporating human feedback, as it reduces the burden on human users.

For addressing limitations, several ideas come to mind. For sparse or inconsistent feedback, active learning techniques that selectively query human feedback only on uncertain or ambiguous states could aid in ameliorating this limitation. This could reduce the amount of feedback needed and improve learning efficiency. Feedback could also be crowdsourced and aggregated to provide diverse sources and also increase robustness. For conflicting feedback, majority voting and weighted averaging could be implemented. In terms of helping the agent generalize across environments when faced with different tasks or settings, meta-learning techniques could be useful in allowing the agent to learn how to adapt both to the different human preferences and variable task constraints across environments.

In conclusion, this study demonstrates the successful implementation of a reinforcement learning algorithm with human feedback for the CartPole game. The results show a significant improvement in user satisfaction, with the mean enjoyment rating increasing from 3.396 to 5.144 when human feedback was incorporated. This improvement was statistically significant, with a t-test p-value of 4.91610^{-9}. However, the variance in user ratings did not change significantly between the baseline and human feedback implementations. The analysis revealed several key insights and limitations: **(1)** The success of the model depends on accurate prior and likelihood assumptions in the Bayesian reward model, **(2)** There is a risk of overfitting to sparse or inconsistent human feedback, and **(3)** The current implementation may have limited generalizability to more complex environments or different tasks. Future directions for research include: **(1)** Implementing advanced RL algorithms such as Q-learning and Deep Q-Networks (DQNs) to enhance performance and adaptability, **(2)** Exploring hybrid approaches that combine RLHF with other RL techniques, and **(3)** Utilizing active learning techniques to selectively query human feedback on uncertain states. These advancements could potentially address the current limitations and further improve the alignment of AI systems with human preferences and values across diverse applications.

References

[1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, Nature **529**, 484 (2016).

[2] H. V. Hasselt, A. Guez, and D. Silver, in *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30 (2016) pp. 1–8.

[3] M. Towers, A. Kwiatkowski, J. Terry, J. Balis, G. De Cola, T. Deleu, M. Goulao, A. Kallinteris, M. Krimmel, A. KG, and R. Perez-Vicente, arXiv preprint arXiv:2407.17032 (2024).

[4] A. G. Barto, R. S. Sutton, and C. W. Anderson, IEEE Transactions on Systems, Man, and Cybernetics **SMC-13**, 834 (1983).

[5] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, in *Advances in Neural Information Processing Systems*, Vol. 30 (2017).

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, in *arXiv preprint arXiv:1707.06347* (2017).

[7] D. A. Pomerleau, in *Proceedings of the 1st International Conference on Neural Information Processing Systems (NeurIPS)* (1989) pp. 305–313.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* (MIT Press, 2018).

[9] A. Y. Ng and S. J. Russell, in *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)* (2000) pp. 663–670.

[10] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, arXiv preprint arXiv:2203.02155 (2022).

[11] R. J. Williams, Machine Learning **8**, 229 (1992).

[12] R. S. Sutton, D. A. McAllester, E. Mansimov, and D. Precup, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 12 (1999) pp. 1057–1063.

[13] P. N. Parkhi, H. Ganwani, M. Anandani, B. Hambarde, P. Agrawal, G. Kaur, P. Pande, and D. Verma, Journal of Electrical Systems **20**, 457 (2024).